

1. Objectives:

1. To understand the basic concept of a Singly Linked List.
2. To learn the operations of insertion at the beginning, at the end, deletion of the first node, last node, and traversal.
3. To analyze the advantages, working mechanism, and limitations of Singly Linked Lists compared to linear data structures like arrays.

2. Introduction:

- **Linked List:** A linked list is a non-contiguous linear data structure in which elements are stored in non-contiguous memory locations. Unlike arrays, linked lists use dynamic memory allocation and connect elements using pointers. Each element in a linked list is called a node, and a node consists of two parts: data and a pointer to the next node. The first node is called the head node, and we can traverse the whole list using this head and next links. There are 3 types of linked list :

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List
4. Doubly Circular Linked List

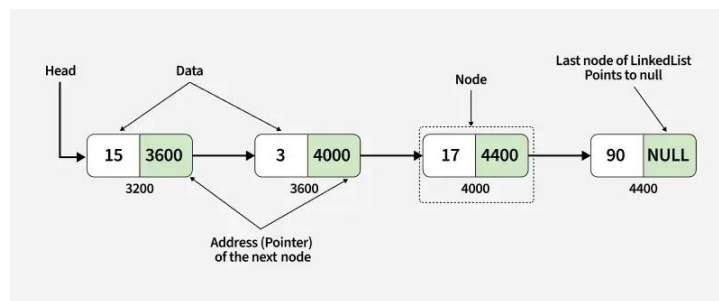


Fig:5.1: Linked List

- **Singly Linked List:** Singly Linked List is the simplest type of linked list where each node contains a value and a pointer that points to the next node in the sequence. The first node is referenced by a pointer called start or head, and the last node points to NULL, indicating the end of the list. In a singly linked list, each node consists of two parts: data and a pointer to the next node. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.

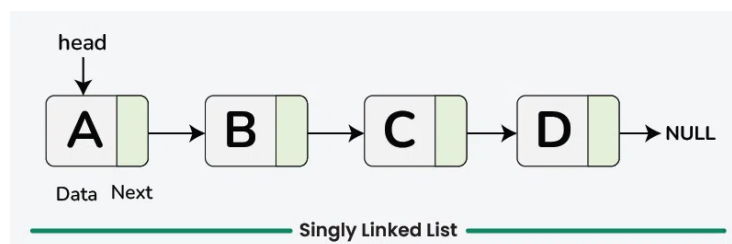


Fig:5.2: Singly Linked List

To insert a new node at the front, a new node is added by creating a node and adjusting pointers. For insertion at the beginning, the new node's next pointer is set to the current head, and then the head is updated to the new node. For insertion at the end, we traverse the list until the last node and set its next pointer to the new node.

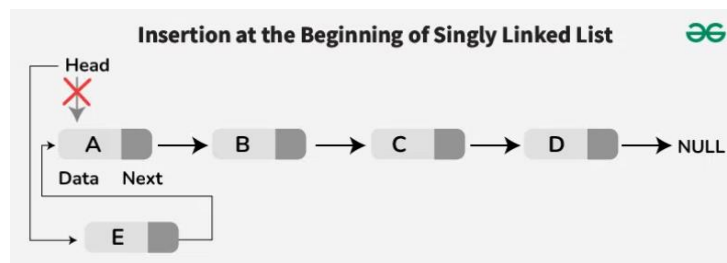


Fig:5.3: Insertion at start

To delete a node, pointers must be updated carefully. For deleting the first node, the head is moved to the next node. For deleting the last node, we traverse to the second last node and set its next pointer to NULL, removing the final node from the list.

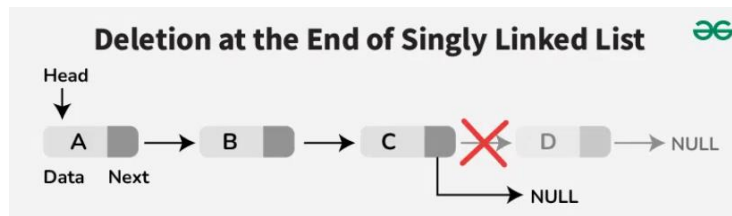


Fig:5.3: Insertion at start

3. Procedure

Experiment 1: Linked List

Input:

- k – Integer value to be inserted into the linked list
- start – Pointer to the first node of the list

Output:

- Printed values of all nodes in the singly linked list

Algorithm:

InsertEnd(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. start $\leftarrow n$
4. **else**
5. $p \leftarrow \text{start}$
6. **while** $p \rightarrow \text{next} \neq \text{NULL}$ **do**
7. $p \leftarrow p \rightarrow \text{next}$
8. $p \rightarrow \text{next} \leftarrow n$

Traverse(start):

1. $p \leftarrow \text{start}$
2. **while** $p \neq \text{NULL}$ **do**
3. **print** $p \rightarrow \text{val}$
4. $p \leftarrow p \rightarrow \text{next}$
5. **print** newline

Experiment 2: Singly Linked List Insert

Input:

- k – Integer value to be inserted

- start – Pointer to first node of the list

Output:

- Printed values of the linked list after insertion

Algorithm:

InsertEnd(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. start $\leftarrow n$
4. **else**
5. $p \leftarrow \text{start}$
6. **while** $p \rightarrow \text{next} \neq \text{NULL}$ **do**
7. $p \leftarrow p \rightarrow \text{next}$
8. $p \rightarrow \text{next} \leftarrow n$

InsertBeginning(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. start $\leftarrow n$
4. **else**
5. $n \rightarrow \text{next} \leftarrow \text{start}$
6. start $\leftarrow n$

Traverse(start):

1. $p \leftarrow \text{start}$
2. **while** $p \neq \text{NULL}$ **do**
3. **print** $p \rightarrow \text{val}$
4. $p \leftarrow p \rightarrow \text{next}$
5. **print** newline

Experiment 3: Singly Linked List Delete

Input:

- k – Integer value to be inserted
- start – Pointer to first node of the list

Output:

- Updated linked list after insertion and deletion operations

Algorithm:

InsertEnd(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** start == NULL **then**
3. start $\leftarrow n$
4. **else**
5. $p \leftarrow \text{start}$
6. **while** $p \rightarrow \text{next} \neq \text{NULL}$ **do**
7. $p \leftarrow p \rightarrow \text{next}$
8. $p \rightarrow \text{next} \leftarrow n$

InsertBeginning(start, k):

1. $n \leftarrow \text{new Node}(k)$
2. **if** $\text{start} == \text{NULL}$ **then**
3. $\text{start} \leftarrow n$
4. **else**
5. $n \rightarrow \text{next} \leftarrow \text{start}$
6. $\text{start} \leftarrow n$

DeleteFirst(start):

1. $\text{start} \leftarrow \text{start} \rightarrow \text{next}$

DeleteLast(start):

1. $j \leftarrow \text{start}$
2. **while** $j \rightarrow \text{next} \rightarrow \text{next} \neq \text{NULL}$ **do**
3. $j \leftarrow j \rightarrow \text{next}$
4. $j \rightarrow \text{next} \leftarrow \text{NULL}$

Traverse(start):

1. $p \leftarrow \text{start}$
2. **while** $p \neq \text{NULL}$ **do**
3. **print** $p \rightarrow \text{val}$
4. $p \leftarrow p \rightarrow \text{next}$
5. **print** newline

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Linked List

```
#include <bits/stdc++.h>
using namespace std;
class Node{
public:
    int val;
    Node *next;

    Node(int k){
        val = k;
        next = NULL;
    }
};

class List{
public:
    Node *start = NULL;
    void ins(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            Node *p = start;
            while (p->next != NULL){
                p = p->next;
            }
            p->next = n;
        }
    }

    void traverse(){
        Node *p = start;
        while (p != NULL){
```

```

        cout << p->val << "\\t";
        p = p->next;
    }
    cout << endl;
}
};

int main(){
    List t;
    t.ins(5);
    t.ins(10);
    t.ins(3);
    t.ins(6);
    t.traverse();
    return 0;
}

```

Table 5.1: Experiment 1 output

Input	Expected Output				Actual Output			
	5	10	3	6	5	10	3	6

Experiment 2: Singly Linked List Insert

```

#include <bits/stdc++.h>
using namespace std;
class Node{
public:
    int val;
    Node *next;

    Node(int k){
        val = k;
        next = NULL;
    }
};

class List{
public:
    Node *start = NULL;
    void ins(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            Node *p = start;
            while (p->next != NULL){
                p = p->next;
            }
            p->next = n;
        }
    }

    void insLast(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            n->next = start;
            start = n;
        }
    }

    void traverse(){
        Node *p = start;
        while (p != NULL){
            cout << p->val << "\\t";
            p = p->next;
        }
        cout << endl;
    }
};

```

```

int main(){
    List t;
    t.ins(5);
    t.ins(10);
    t.ins(3);
    t.ins(6);
    t.insLast(9);
    t.traverse();
    return 0;
}

```

Table 5.2: Experiment 2 output

Input	Expected Output					Actual Output				
	9	5	10	3	6	9	5	10	3	6

Experiment 3: Singly Linked List Delete

```

#include <bits/stdc++.h>
using namespace std;
class Node{
public:
    int val;
    Node *next;

    Node(int k){
        val = k;
        next = NULL;
    }
};

class List{
public:
    Node *start = NULL;
    void ins(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            Node *p = start;
            while (p->next != NULL){
                p = p->next;
            }
            p->next = n;
        }
    }

    void insLast(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            n->next = start;
            start = n;
        }
    }

    void deleteFirst(){
        start = start->next;
    }

    void deleteLast(){
        Node *j = start;
        while (j->next->next != NULL){
            j = j->next;
        }
        j->next = NULL;
    }
}

```

```

void traverse(){
    Node *p = start;
    while (p != NULL){
        cout << p->val << "\t";
        p = p->next;
    }
    cout << endl;
}

};

int main(){
    List t;
    t.ins(5);
    t.ins(10);
    t.ins(3);
    t.ins(6);
    t.insLast(9);
    t.traverse();
    t.deleteFirst();
    t.traverse();
    t.deleteLast();
    t.traverse();
    return 0;
}

```

Table 5.3: Experiment 3 output

Input	Expected Output					Actual Output				
	9	5	10	3	6	9	5	10	3	6
	5	10	3	6		5	10	3	6	
	5	10	3			5	10	3		

6. Discussion & Conclusion:

In this lab, we successfully learned about Linked List and Singly Linked List. We performed insertion at the end, insertion at the beginning, deletion of the first node, deletion of the last node, and traversal of the list. We learned how nodes are dynamically created using pointers and how they are linked together using the next reference. We observed that insertion at the beginning is more efficient than insertion at the end because it requires fewer operations. Deletion operations also required careful pointer manipulation to maintain list integrity. Compared to arrays, linked lists provide flexibility in memory usage and dynamic size management but require additional memory for storing pointers and careful handling to avoid logical errors. Overall, this experiment improves our understanding of dynamic data structures and pointer-based memory management in C++.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/linked-list-data-structure/> accessed at 9.25 AM on 4th March 2026.
2. <https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/> accessed at 10.40 PM on 4th March 2026.
3. <https://www.geeksforgeeks.org/dsa/insertion-in-linked-list/> accessed at 12.35 PM on 4th March 2026.